

PUP RIDES: RIDE BOOKING SYSTEM USING PYTHON

A Project Report Submitted in Partial Fulfillment of the
Requirements for CMPE 103: Object-Oriented Programming

Presented to: Prof. Godofredo Talamayan Avena

Prepared By:

Domingo, Franco Luis

Mesias, Lewell

Gacu, Laiza May M.

Pangandaman, Najer D.

Luna, Sybella Llorin B.

Sabida, Ghale Anne

Mamaril, Jayson Cris L.

San Luis, Ghani Regina Gold B.

Mejia, Aayel

Tatlonghari, Jan Druelle

June 15, 2026

Abstract

This project details the development of PUP Rides, a desktop-based Ride Booking System application implemented in Python 3 using Visual Studio Code. Designed to streamline transportation management, the system enables users to seamlessly book rides, view current bookings, and cancel requests. The core architecture leverages Object-Oriented Programming (OOP) principles—including inheritance, encapsulation, and polymorphism—to manage distinct vehicle types (Cars, Vans, and Motorcycles) and dynamically compute individual fare rates. To ensure data persistence across sessions, Python's file-handling capabilities were integrated to automatically save and load booking records to a local storage file. Furthermore, a user-friendly Graphical User Interface (GUI) was developed using Tkinter, bridging the gap between complex application logic and intuitive user interaction. Ultimately, the project demonstrates that combining modular OOP design with reliable file persistence and clean GUI forms yields a highly scalable, maintainable, and practical software solution.

Keywords: Object-Oriented Programming, Python, Tkinter, Data Persistence, Ride Booking System

Table of Contents

Title Page	i
Abstract	ii
I. Introduction	5-8
• 1.1 Background of the Study	5
• 1.2 Problem Statement	5-6
• 1.3 Project Objectives	6
• 1.4 Scope and Limitations	7
<i>1.4.1 In-Scope Deliverables</i>	7
<i>1.4.2 Out-of-Scope Limitation</i>	7-8
II. Methodology	8-10
• 2.1 Research Design	8
• 2.2 Data Collection and Management Methods	9
• 2.3 Tools Used	10
III. System Design and Implementation	10-13
• 3.1 System Overview	10-11
• 3.2 Key Components	11-12
• 3.3 Implementation Process	12-13
IV. Testing and Results	13-17
• 4.1 Testing Methods	13
• 4.2 Test Results (Test Matrix)	14-16
• 4.3 Analysis of Results	16-17
<i>4.3.1 Architectural Reliability of OOP Core</i>	17

4.3.2 <i>UI Abstraction and Input Validation Safety</i>	17
4.3.3 <i>Data Integrity via Sequential File Operations</i>	17
V. Conclusion and Recommendations	17-20
• 5.1 Summary of Findings	17
• 5.2 Recommendations	19
5.2.1 <i>Enhance Dynamic Pricing and Feature Logic</i>	19
5.2.2 <i>Refactor User and Driver Market Workflows</i>	19
5.2.3 <i>Integrate Automated Communication and Reporting Utilities</i>	19-20
5.2.4 <i>Transition to a Persistent Relational Database Architecture</i>	20
5.2.5 <i>Improve Code Structure and Quality Assurance</i>	20

I. Introduction

1.1 Background of the Study

Modern ride-booking systems have revolutionized urban transportation by seamlessly connecting passengers with diverse vehicular services. From a software engineering perspective, designing and implementing an on-demand booking system serves as an excellent benchmark for mastering complex data management, software architecture, and user experience design.

The project, titled PUP Rides, utilizes Python 3 within the Visual Studio Code integrated development environment to build a fully functional, desktop-based ride-booking application. Rather than employing sequential or procedural scripting, the application heavily relies on Object-Oriented Programming (OOP) paradigms. By mapping real-world entities—such as distinct vehicle types and transactional customer records—into structured Python classes, the application ensures strict modularity, maintainability, and clean code organization.

1.2 Problem Statement

Developing an efficient, on-demand transportation platform requires managing multiple interdependent system components simultaneously. The primary challenges addressed by this project include:

- **Fleet Diversification:** Dynamically managing a fleet composed of distinct vehicle classes (Cars, Vans, and Bikes), each possessing unique passenger capacities, operational constraints, and varying fare calculations.
- **Data Volatility:** Preventing the loss of transactional data during application runtime without incurring the overhead of complex database server setups.

- User Abstraction: Bridging the gap between intricate backend algorithmic logic (such as polymorphic fare computations) and an intuitive, non-technical end-user interface.

1.3 Project Objectives

The primary objective of this study is to design, develop, and deploy the PUP Rides application. Specifically, the project aims to achieve the following technical goals:

- Implement Vehicle Management via OOP: Apply fundamental OOP concepts like classes and inheritance by engineering a base vehicle class that smoothly derives specialized subclasses for Cars, Vans, and Bikes.
- Apply Polymorphism for Dynamic Fare Calculations: Implement method overriding across the derived vehicle subclasses to automatically compute unique trip costs based on specific vehicle variables, base rates, and efficiency metrics.
- Develop Core Booking Features: Construct a transactional backend logic that generates unique booking IDs, captures route variables, lists active itineraries, and processes cancellations seamlessly.
- Integrate File Operations for Data Persistence: Implement Python file-handling routines (`open()`, `read()`, `write()`) to establish local text-based data persistence, ensuring data automatically saves during updates and reloads upon system initialization.
- Design an Intuitive Graphical User Interface (GUI): Build a responsive desktop front-end using the Tkinter library featuring clean input forms, operational event-driven buttons, and structured data display tables.

1.4 Scope and Limitations

1.4.1 In-Scope Deliverables

The boundary of this project is confined to the architecture, design, and local deployment of a desktop ride-booking system. The specific deliverables include:

- **Vehicle Archetypes:** Integration of three distinct vehicle classifications: Cars, Vans, and Bikes, each bound to independent cost calculation factors.
- **Transactional Controls:** UI mechanisms and logic for booking placement, comprehensive record listings, and ride cancellations.
- **Data Persistence Layer:** Automated sequential file I/O operations that save session states locally whenever a database mutation occurs (additions or cancellations).
- **Desktop Interface Layout:** A comprehensive Tkinter graphical shell optimized for layout structure, input fields, forms, and event handling.

1.4.2 Out-of-Scope Limitations

To maintain project focus and stay within development constraints, the following advanced architectural features are explicitly excluded from this study:

- **Geospatial Tracking:** The application relies entirely on text-based text strings for start and destination locations rather than live Global Positioning System (GPS) or map API mapping integrations.

- Financial Transaction Gateways: The system calculates and displays the theoretical financial cost of a fare but does not connect to external electronic payment gateways, banks, or credit card processors.
- Distributed Cloud Architecture: The software operates strictly as a standalone, localized application on the host execution machine and does not synchronize with remote cloud databases or web servers.

II. Methodology

2.1 Research Design

This project utilizes an Applied Software Development Research Design, following an iterative development model. The design focuses on solving a practical problem—managing vehicle bookings—by constructing a software artifact named PUP Rides.

The system's architecture is deeply rooted in Object-Oriented Programming (OOP) paradigms to ensure high modularity, reusability, and ease of maintenance. The research design is structured into three progressive layers:

- Data Modeling Layer: Creating a robust, hierarchical class structure to encapsulate vehicle properties and dynamic behaviors.
- Data Persistence Layer: Designing a lightweight, reliable system to bridge memory states with local storage files.
- User Interaction Layer: Developing a visual interface to abstract backend complexities away from the end user.

2.2 Data Collection and Management Methods

Because this is an applied software implementation project rather than an empirical study, data collection and data management focus heavily on system-generated runtime records and transactional user data. Data collection begins dynamically at the user interaction level, where the system captures input variables through customized Tkinter graphical forms. These gathered data attributes include user details, selected vehicle classifications (Car, Van, or Bike), and specific trip parameters required by the system's backend formulas to calculate pricing.

To manage this information and prevent data volatility, a native file-handling mechanism acts as the data persistence layer. The system utilizes sequential file operations to automatically write active session data to a local text or flat file whenever a user adds or cancels a ride. Conversely, during application initialization, the system executes a startup routine that opens this local file, parses the stored strings, and dynamically reconstructs the data into operational objects within the computer's memory. This ensures that user records remain perfectly preserved between independent execution cycles without relying on complex, external database server dependencies.

2.3 Tools Used

The development, version tracking, and management of the PUP Rides application relied on the following software stack, tools, and libraries:

Tool / Technology	Purpose in the Project
Python 3	The primary, high-level programming language used to implement the backend logic, OOP structures, and file operations.
Visual Studio Code (VS Code)	The Integrated Development Environment (IDE) utilized for code authoring, debugging, and environment management.
Tkinter Library	Python's standard GUI toolkit, used to design forms, buttons, and data display areas for an intuitive user experience.
Git	The distributed version control system used locally to track code revisions, manage development branches, and maintain a detailed history of changes.
GitHub	The cloud-based hosting service used for remote repository storage, backup preservation, and codebase version management.
Built-in File I/O	Python's native file-handling functions used to manage local data persistence without the need for external database dependencies.

III. System Design and Implementation

3.1 System Overview

The PUP Rides application is a ride booking program built in Python and made exclusively for use within the Polytechnic University of the Philippines. It is a transportation

coordination tool that links passengers and drivers inside the university environment. On opening the application, the user is required to pick his/her role from two available roles, namely the Passenger role or the Driver role. If the user chooses the Passenger role, then they can request for a ride, and if the user chooses the Driver role, then they will be the ones accepting rides requested by passengers. The PUP Rides system uses the distinct colors of PUP which symbolizes where it originates from.

3.2 Key Components

The system is made up of several interlinked modules that are responsible for supporting the entire process of ride booking. Firstly, the system features an Authentication Module that deals with account registration and log in services offered for passengers and drivers separately using two distinct but structurally similar interfaces. New Users need to create an account indicating their full names, usernames and password, then you can proceed to sign in when you have already created an account. Meanwhile, the driver login portal works independently as its own platform in order to highlight the differences in roles.

Secondly, there is a Passenger Dashboard that is used to provide the main functionality of ride booking within the system. The passengers have the privilege to select what type of vehicle they like to use through selecting among Car, Van, and Bike, accompanied by the capacity restrictions as well as pricing policies concerning them. Passengers may select the location where they will begin their ride, the place they intend to go, the distance in kilometers that they wish to cover, and the number of riders. Another function of the system is the Fare Estimator which estimates the fare that the passenger will pay depending on the vehicle type and the distance,

wherein there is a surge charge of 1.5 times the base fare in the rush hours from 7:00 to 9:00 AM and from 5:00 to 8:00 PM. In terms of pricing structure, the vehicles follow the format of Base Fare + Distance. The Cars will be charged with 40 PHP as base fare plus 14 PHP per kilometer having four capacity while Vans will be charged 80 PHP base fare plus 20 PHP per kilometer with 10 capacity passengers.

3.3 Implementation Process

The system was made as a desktop graphical user interface application using Python, with the Tkinter library serving as the main framework for the windows and widgets that are used in the system. The interface was structured around multiple window frames, with each functional screen rendered as a separate window or panel to maintain clear navigation flow between the role selection screen, authentication screens, and the respective dashboards. Management of data, particularly login details and booking details, was done using file-based or in-memory data management in accordance with the level of a project involving a programming language such as Python.

The logic for determining the fare was done by developing a calculation algorithm where values like mode of transport, distance traveled, and time of day were the input values and base fare, fare per kilometer, and surcharge were used to calculate the current fare to be shown to the user. The interface design was achieved by use of Tkinter widgets' styling capabilities in ensuring the use of the maroon and gold color combination in all frames, buttons, labels, and entry fields. Iconic features were included in order to enhance the navigability of the interface.

The "About" window in this application displays the name and version of the program with the names of its creators.

IV. Testing and Results

This chapter details the testing methodologies, execution scripts, and structural results used to validate the functional integrity of the PUP Rides application. It outlines the specific evaluation criteria applied to ensure the system behaves reliably under normal and exceptional conditions.

4.1 Testing Methods

The software artifact was subjected to rigorous testing procedures to ensure that both the backend Object-Oriented Programming (OOP) logic and the Tkinter Graphical User Interface (GUI) operate seamlessly together. Two primary software testing methods were used:

- **Functional Unit Testing:** Isolated testing applied directly to the backend classes (Car, Van, Bike) to verify that vehicle properties, capacity limits, and polymorphic fare calculation formulas compute mathematical outputs accurately based on input distance.
- **Black-Box Integration Testing:** End-to-end testing performed through the Tkinter user interface. This method assesses how components interact together—ensuring that clicking interface buttons correctly invokes backend functions, updates the in-memory array, triggers file I/O serialization, and Refreshes the visual display grid.

4.2 Test Results

A comprehensive test suite containing critical runtime scenarios was executed within the Visual Studio Code development environment. The operational results are recorded in the test matrix below:

Test ID	Functional Component	Input / Action Trigger	Expected System Behavior	Actual Result	Status
TC-01	OOP Inheritance & Instantiation	Select "Bike" archetype and input route strings.	System instantiates a Bike object with a max capacity of 1 passenger.	Object instantiated successfully with accurate local attributes.	PASSED
TC-02	Polymorphic Fare Computation	Input a distance of 10km for "Car" and "Van" options.	Car and Van calculate distinct, unique fares based on overridden	Car and Van returned distinct, mathematically correct rates.	PASSED

			subclass formulas.		
TC-03	Interface Form Submission	Click "Book Ride" button with blank text input fields.	GUI halts execution and displays an error dialog requesting required fields.	Input validation triggered; error dialog blocked empty submission.	PASSED
TC-04	Persistent Data Auto-Save	Complete a successful ride booking transaction.	System open text/flat file, serializes the object, and writes records locally.	Local file updated instantly with flattened transaction string.	PASSED
TC-05	Transactional State Erasure	Select an active booking record and	Object is removed from runtime	Record removed from memory array and local	PASSED

		click "Cancel Ride".	memory array and deleted from the local text file.	storage simultaneously.	
TC-06	Boot-Phase Deserialization	Launch the application with an existing text file in directory.	Startup script reads, parses, and reinstantiate s active objects into the UI grid.	All previous booking records successfully populated the UI table on boot.	PASSED

4.3 Analysis of Results

An analysis of the testing data yields several key insights regarding the structural performance and stability of the PUP Rides software application:

4.3.1 Architectural Reliability of OOP Core

The successful execution of TC-01 and TC-02 proves that the underlying class hierarchy is robust. Because polymorphism dynamically routes the distance parameter to the correct overridden subclass method at runtime, fare calculations remain

mathematically precise and isolated. This prevents logic errors where a passenger might be billed a premium Van rate for a standard Bike ride.

4.3.2 UI Abstraction and Input Validation Safety

Data gathered from TC-03 confirms that the Tkinter interface does not simply pass raw user input straight to the backend. The integration of validation guardrails prevents empty or corrupt data fields from breaking the application's runtime loop, thereby ensuring software stability even when handling unpredictable user inputs.

4.3.3 Data Integrity via Sequential File Operations

The execution results of TC-04, TC-05, and TC-06 validate the effectiveness of the local file-handling persistence layer. Linking data mutation triggers (bookings and cancellations) directly to automated file-write sequences guarantees that the local text file mirrors active memory perfectly.

The successful restoration of records during cold startups confirms that the string-splitting and parsing logic converts raw flat-file text back into complex programming objects cleanly, achieving reliable data persistence without the overhead of a formal database system.

V. Conclusion and Recommendations

This chapter concludes the study by summarizing the core technical findings derived from the development and testing of the PUP Rides application. It also provides actionable recommendations for future architectural and functional improvements.

5.1 Summary of Findings

The development, deployment, and testing of the PUP Rides application successfully demonstrate how Object-Oriented Programming (OOP) concepts can be applied to solve real-world software problems. Through rigorous testing within the Visual Studio Code environment, the following key findings were established:

- **Object-Oriented Structuring:** Organizing the application into a strict class hierarchy allowed for seamless management of diverse vehicle fleets. Utilizing single inheritance effectively eliminated code redundancy, allowing the Car, Van, and Bike subclasses to inherit shared attributes from a single base Vehicle class, ensuring code maintainability.
- **Polymorphic Efficiency:** Implementing polymorphism through runtime method overriding proved to be an exceptional approach for dynamic fare calculations. The system successfully executed unique cost formulas for each vehicle type automatically based on specific subclass criteria, proving that abstract interfaces can manage varied mathematical operations reliably.
- **Reliable Data Persistence:** The integration of native Python file-handling utilities successfully resolved the issue of volatile program memory. The sequential read/write pipelines proved that automated local serialization can prevent data loss during ride creations or cancellations, keeping session states consistent without database overhead.
- **User Interface Accessibility:** Constructing the Graphical User Interface (GUI) via the Tkinter library significantly enhanced user interaction. The form fields, action triggers, and display grids abstracted the system's backend complexity, allowing non-technical users to book rides, track records, and execute cancellations seamlessly without interacting with raw code.

5.2 Recommendations

Based on the successful completion of the PUP Rides prototype and the specific limitations encountered during its execution, the following recommendations are suggested for future iterations of the software system:

5.2.1 Enhance Dynamic Pricing and Feature Logic

Future iterations should shift from static pricing variables to dynamic, demand-based calculation models where fares automatically scale based on simulated real-time booking volume. Additionally, the administrative coupon/promotional modules should be refactored to support conditional validation rules, such as rigid expiration dates and maximum usage limits per customer profile.

5.2.2 Refactor User and Driver Market Workflows

To transition the software into a highly realistic marketplace simulation, the current pseudo-random driver allocation mechanism should be replaced with an interactive driver acceptance or rejection workflow. Furthermore, expanding the transactional feedback loop by introducing mutual passenger-driver rating forms is recommended to deepen system logic.

5.2.3 Integrate Automated Communication and Reporting Utilities

The functional utility of the software can be extended by utilizing Python's built-in `smtplib` library to dispatch automated email booking confirmations directly to users upon transaction approval. Similarly, the local reporting module should be

upgraded to enable data conversion features, allowing passengers to export their complete ride histories directly into a standard PDF format.

5.2.4 Transition to a Persistent Relational Database Architecture

While sequential flat-file persistence successfully satisfied the local storage constraints of this prototype, it is highly recommended to replace flat files with an integrated SQLite database engine. Because SQLite is natively supported by Python, this transition is highly achievable and would allow the application to scale efficiently, support multi-table relational queries, and handle dynamic inventory tasks such as administrative updates to active fleets.

5.2.5 Improve Code Structure and Quality Assurance

As the user interface expands, the primary window script should be decomposed into smaller, modular UI components to prevent code bloat and strictly preserve the Separation of Concerns (SoC) principle. Finally, integrating automated unit testing modules (such as Python's native unittest framework) is strongly recommended to quickly verify that polymorphic cost formulas remain mathematically accurate during future codebase upgrades.